Application SQL Server DBA Interview Preparation and Answer Guide

Aligned to current Microsoft Learn guidance for SQL Server administration, replication, migration, performance troubleshooting, archiving, and security.

1. Purpose and Scope

- This doc standardizes the answers an Application SQL Server DBA candidate or interviewer should use for a production support, migration, replication, and Azure-focused interview.
- Every technical claim is checked against current Microsoft Learn documentation before inclusion.
- The document covers SQL Server administration, replication, archiving, performance troubleshooting, Azure migration, high availability limits by edition, regional data security, and change management.
- Use this as a rehearsal script. Read each question, answer in your own words using the bullet points, then check the reference link.

2. DBA Background and Experience Summary

Q1. Tell me about your SQL Server DBA background.

- State total years of experience and the mix of production support, application DBA work, migrations, and performance tuning.
- Name the areas you own: high availability, security, change management, and database maintenance.
- Mention coordination with application, development, infrastructure, and enterprise DBA teams.
- Describe your production change approach in one line: validate in non-production, document rollback, execute in an approved window, validate, then monitor.

Keep this answer under two minutes. Save the deep technical detail for the follow-up questions below.

3. SQL Server Migration

Q2. What types of SQL Server migrations have you performed?

- Group migrations into three categories: version upgrades, platform or infrastructure migrations, and cloud migrations.
- Before migration, inventory databases, SQL Agent jobs, logins and SIDs, linked servers, credentials, certificates, SSIS packages, and application connection strings.
- Verify version and edition compatibility, database compatibility level, deprecated features, HA/DR dependencies, and storage sizing.
- Capture a performance and workload baseline before you touch anything.
- Choose the migration mechanism based on database size and allowed downtime: backup and restore, log shipping style restore sequencing, availability group seeding, or an Azure migration service.
- Run a test migration and full application validation before the production cutover.
- During cutover: pause or control application writes, run the final sync, validate data and consistency, redirect the application, run smoke tests, and monitor.
- Keep a documented rollback plan for every migration, no exceptions.

Reference: Microsoft Learn, SQL Server to Azure SQL Managed Instance Migration Guide, learn.microsoft.com/data-migration/sql-server/managed-instance/guide

Q3. What is the largest database you have managed, and how does size change your approach?

- Give the size, for example roughly 2 TB, then explain the operational impact rather than stopping at the number.
- At that scale you pay close attention to backup and restore duration, RPO and RTO, transaction log growth, storage latency and throughput.
- Index maintenance strategy, statistics freshness, integrity checks, HA/DR design, and capacity planning all become harder at scale.
- State clearly that size alone does not define complexity. Workload pattern, change rate, and I/O profile matter as much as the gigabyte count.

4. SQL Server Replication

Q4. What SQL Server replication types do you know, and when do you use each?

- Snapshot replication distributes a full point-in-time copy. Use it when data changes infrequently or a periodic full refresh is acceptable.
- Transactional replication takes an initial snapshot, then continuously propagates committed changes from Publisher through Distributor to Subscribers. Use it for low-latency, one-directional synchronization.
- Merge replication allows changes at multiple nodes with later synchronization, including conflict detection and resolution.
- Selection criteria: direction of sync, acceptable latency, connectivity, conflict handling needs, topology, database size, and operational complexity.

Reference: Microsoft Learn, Types of Replication, learn.microsoft.com/sql/relational-databases/replication/types-of-replication

Q5. Is snapshot replication bidirectional?

- No. Snapshot replication is a one-way Publisher-to-Subscriber distribution.
- It generates a point-in-time copy of the published schema and data and pushes it to Subscribers.
- Do not present it as a two-way synchronization technology in an interview.

Reference: Microsoft Learn, Snapshot Replication, learn.microsoft.com/sql/relational-databases/replication/snapshot-replication

Q6. Can transactional replication replicate only selected tables?

- Yes. Replication works at the article level, so you publish only the tables you need instead of the whole database.
- Every table published as an article in standard transactional replication must have a primary key.
- Row and column filters narrow the published data further when only a subset of columns or rows is required.
- Publishing only the required data is also a Microsoft-documented performance best practice for reducing distribution overhead.

Reference: Microsoft Learn, Publish Data and Database Objects; Enhance General Replication Performance, learn.microsoft.com/sql/relational-databases/replication

Q7. Does snapshot replication require a primary key on the published table?

- No. Snapshot replication does not require a primary key.
- Transactional replication does require a primary key on published tables.

- Merge replication does not require an existing primary key, but replication adds its own tracking columns and has separate considerations.

Reference: Microsoft Learn, Frequently Asked Questions for Replication Administrators, learn.microsoft.com/sql/relational-databases/replication/administration/frequently-asked-questions-for-replication-administrators

5. Data Archiving Strategy (25 Years of Data)

Q8. The business has 25 years of data and wants only the last 5 years active. How do you design the archiving project?

- Do not start with filegroups or partitioning. Start with requirements.
- Identify which tables hold historical data and the retention period required per data domain.
- Map relationships and foreign key dependencies before you move a single row.
- Confirm whether archived data still needs to be queryable, and what response time is acceptable for archive lookups.
- Check compliance and legal retention requirements, current table sizes, and annual growth rate.
- Review which application queries touch historical records today.
- Define the archive backup, security, and disaster recovery requirements separately from the live system.
- For the design: if a large table has a reliable date key, partitioning can simplify lifecycle management. Otherwise, maintain a separate archive database and move qualifying rows in controlled batches.
- Archive parent and child rows together so referential integrity stays valid.
- Batch the moves to control transaction log growth, locking, and blocking.
- Validate row counts and checksums, and confirm application behavior, before deleting anything from the active database.
- Automate the retention job on a schedule, for example monthly, and monitor runtime, blocking, log usage, and archive growth.

Q9. Is table partitioning available in SQL Server Standard Edition?

- Confirm the exact SQL Server version and build before answering, because edition capability for partitioning has changed across releases.
- Never design an archival strategy around an assumed Enterprise-only limitation. Verify it on the actual server first.

```
SELECT
   SERVERPROPERTY('Edition')       AS Edition,
   SERVERPROPERTY('EngineEdition')  AS EngineEdition,
   SERVERPROPERTY('ProductVersion') AS ProductVersion,
   SERVERPROPERTY('ProductLevel')   AS ProductLevel;
```

Reference: Microsoft Learn, Editions and supported features of SQL Server, learn.microsoft.com/sql/sql-server/editions-and-components-of-sql-server

6. Performance Troubleshooting

Q10. The application is slow after a data move. Walk through your troubleshooting sequence.

- Step 1, establish the symptom. Is the whole application slow or one function? When did it start? What changed? Is it constant or intermittent?

- Step 2, check SQL Server and infrastructure health: CPU, memory pressure, storage latency, I/O throughput, connection pressure, blocking, waits, transaction log pressure, and error logs.
- Step 3, identify expensive or regressed queries using Query Store, DMVs, and Extended Events.
- Step 4, check blocking and wait types to see whether sessions wait on locks, CPU, I/O, memory, or network.
- Step 5, examine the actual execution plan for poor cardinality estimates, inefficient joins, excessive scans, spills, lookups, implicit conversions, and parameter sensitivity.
- Step 6, check statistics freshness and index usage. Treat missing-index suggestions as input, not an instruction to implement blindly.
- Step 7, compare application behavior against SSMS behavior. Check parameters, SET options, and connection settings if the query behaves differently through the application.
- Step 8, validate the fix outside production.
- Step 9, deploy through the approved change process with a documented rollback.
- Step 10, compare performance after the change against the original baseline.

Interview correction: do not say you will start Profiler to find the slow query. SQL Trace and Profiler are deprecated for new work. Say you will start with Query Store and DMVs, and use Extended Events for targeted capture.

```
-- Confirm Query Store is on and check retention/capture settings
SELECT actual_state_desc, desired_state_desc, query_capture_mode_desc,
    max_storage_size_mb, current_storage_size_mb, stale_query_threshold_days
FROM sys.database_query_store_options;
```

```
-- Top resource-consuming queries from Query Store, last 1 hour
SELECT TOP 20 qt.query_sql_text,
    rs.avg_duration, rs.avg_cpu_time, rs.avg_logical_io_reads,
    rs.count_executions, rs.last_execution_time
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi
   ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(HOUR, -1, SYSUTCDATETIME())
ORDER BY rs.avg_duration DESC;
```

Reference: Microsoft Learn, Best Practices for Monitoring Workloads with Query Store; Understand and Resolve Blocking, learn.microsoft.com/sql/relational-databases/performance/best-practice-with-the-query-store

7. Azure Migration

Q11. How would you migrate an on-premises SQL Server database to Azure?

- Decide the target architecture first, not the migration tool: Azure SQL Database, Azure SQL Managed Instance, or SQL Server on an Azure VM.
- Base the decision on SQL Server feature dependencies, instance-level functionality, required OS control, application changes, HA/DR requirements, database size, performance needs, and operating model.
- Run a migration readiness assessment to surface unsupported features and compatibility issues before committing to a target.
- For Managed Instance, Microsoft currently supports Azure Database Migration Service, the Managed Instance link, Log Replay Service, and native backup and restore, chosen by downtime tolerance and architecture.
- For SQL Server on an Azure VM, backup and restore is a common path, with faster options available when downtime must be minimized.
- Baseline the source, test the migration, validate source versus target, plan cutover and rollback, migrate server-level dependencies, and monitor after cutover.

Reference: Microsoft Learn, SQL Server to Azure SQL Managed Instance Migration Guide, learn.microsoft.com/data-migration/sql-server/managed-instance/guide

Q12. What challenges can occur when migrating even a small 5 GB database to Azure?

- Database size does not determine migration complexity. Do not lead with bandwidth.
- Check for unsupported SQL Server features, T-SQL compatibility differences, and database compatibility level.
- Check logins, users, and SIDs, SQL Agent jobs, linked servers, and CLR or external dependencies.
- Check application connection strings, network latency, firewall rules, and DNS.
- Check encryption and certificates, including TDE, plus collation and database configuration.
- Check expected performance differences on the target tier, and confirm cutover downtime and rollback plan.

Reference: Microsoft Learn, SQL Server to Azure SQL Managed Instance Migration Overview, learn.microsoft.com/data-migration/sql-server/managed-instance/overview

8. Hybrid On-Premises and Cloud Scenarios

Q13. The business has 25 years on-premises and wants only the latest 5 years in Azure. What is your approach?

- Clarify first whether Azure becomes the primary operational system, a read-only reporting copy, or an archive. The answer changes the whole design.
- If only a subset of rows or tables needs to move, evaluate transactional replication with row and column filtering, or a purpose-built ETL or data integration approach.
- If the goal is a full migration to Azure SQL Managed Instance with minimal downtime, evaluate Microsoft migration technologies such as the Managed Instance link, Database Migration Service, or Log Replay Service instead of building custom replication.
- For a rolling five year window, separate two problems: the initial load of the existing five years, and incremental synchronization of new changes afterward.
- Design retention as a separate process from replication. They solve different problems.

- Before production, test synchronization latency, data validation, network throughput, failure and recovery behavior, schema change handling, monitoring, and the cutover itself.

Reference: Microsoft Learn, SQL Server to Azure SQL Managed Instance Migration Overview, learn.microsoft.com/data-migration/sql-server/managed-instance/overview

Q14. Can we move delta data every day or every week?

- Yes, but first determine whether the requirement is scheduled batch movement or continuous synchronization.
- For scheduled batch movement, an ETL or data integration process identifies and transfers deltas on a job schedule.
- For near real time propagation, transactional replication fits supported topologies.
- For a migration scenario into Azure SQL Managed Instance, Microsoft migration technologies continuously synchronize source changes until cutover.
- Base the final architecture on acceptable latency, change volume, conflict requirements, and whether Azure is temporary migration infrastructure or a permanent second copy.

Q15. How expensive is daily synchronization?

- Do not estimate cost from total database size. Measure the change rate instead.
- Key factors: transaction volume, generated transaction log volume, network bandwidth and latency, number and width of replicated articles.
- Also factor Distributor capacity, target write throughput, indexes and constraints on the target, sync frequency, Azure compute, storage and network cost, and monitoring overhead.
- Example: a 2 TB database changing 2 GB a day can be cheaper to synchronize than a 200 GB database changing 100 GB a day.
- Measure daily log generation and peak transaction throughput before designing the synchronization job.

9. Always On Availability Groups and Standard Edition

Q16. Can SQL Server Standard Edition use Always On Availability Groups?

- Standard Edition supports Basic Availability Groups, not the full Enterprise availability group feature set.
- A Basic Availability Group is limited to one database and two replicas, primary and secondary.
- The secondary replica in a Basic Availability Group has no read access. It cannot be used as a readable reporting replica and it cannot take backups.
- Do not tell an interviewer that the Standard Edition secondary can serve read workloads. That is incorrect and Microsoft documentation states the opposite.

Reference: Microsoft Learn, Basic Availability Groups for a Single Database, learn.microsoft.com/sql/database-engine/availability-groups/windows/basic-availability-groups-always-on-availability-groups

Q17. Is there a SQL Server 'Professional Edition'?

- There is no Database Engine edition named Professional. Do not argue over the term.
- Confirm the actual edition directly from the server rather than guessing what the interviewer means.

```
SELECT
   SERVERPROPERTY('Edition')       AS Edition, SERVERPROPERTY('ProductVersion') AS ProductVersion,
   SERVERPROPERTY('ProductLevel')   AS ProductLevel;
```

- This verify-first habit is itself a strong interview signal.

10. Bidirectional Data Synchronization

Q18. Cloud users update 10 tables and those changes must flow back to on-premises. How do you synchronize them?

- First clarify whether both locations can update the same rows, or whether each location owns a distinct subset of rows.
- If one location owns each row, design around that ownership split. It removes most conflict complexity immediately.
- Do not default to legacy transactional replication updatable subscriptions for new development. Microsoft documentation states this feature will be removed in a future version and should be avoided for new work.
- Evaluate supported replication or data integration patterns instead, and design conflict handling explicitly rather than relying on a deprecated built-in mode.
- If multiple locations can legitimately update the same logical record, you need defined ownership rules and an explicit conflict resolution policy, not just a replication topology.

Reference: Microsoft Learn, Updatable Subscriptions for Transactional Replication (feature will be removed, avoid in new development), learn.microsoft.com/sql/relational-databases/replication/transactional/updatable-subscriptions-for-transactional-replication

Q19. Is transactional replication automatically two way?

- No. Standard transactional replication is Publisher to Subscriber only.
- Creating a publication and subscription does not give you bidirectional synchronization by itself.
- Special topologies exist for multi-node updates, but they carry real requirements and limits, and the legacy updatable subscription feature should not be chosen for new development.

Q20. Is replication scheduled or real time?

- It depends on the replication type and how the agents are configured.
- Snapshot generation can run on a schedule because it produces a point-in-time copy.
- Transactional replication is normally used for continuous, low-latency propagation. The Log Reader Agent reads committed transactions from the publication database log, and the Distribution Agent delivers them to Subscribers.
- Agents can run continuously or on a schedule depending on topology and requirement, so do not describe transactional replication as a once-a-day job by default.

Q21. How does transactional replication know what changed?

- It does not scan every table for changed rows.
- The Log Reader Agent reads changes from the Publisher transaction log and writes commands into the distribution database.
- The Distribution Agent applies those commands to the Subscriber, usually through generated stored procedures.
- This log-based mechanism is what gives transactional replication its low latency.

Reference: Microsoft Learn, Specify How Changes Are Propagated (Transactional), learn.microsoft.com/sql/relational-databases/replication/transactional/transactional-articles-specify-how-changes-are-propagated

11. Regional Data Security

Q22. US managers must see only US customers and European managers only European customers, from the same table and database. How do you design it?

- The correct primary control is Row-Level Security, not Dynamic Data Masking.
- Ensure every row carries an authoritative region or tenant attribute, for example RegionId.
- Use a Row-Level Security predicate so each security context only sees its own region.
- Authenticate users through a controlled identity or role model, and enforce the restriction at the database layer, not only in the application.
- Dynamic Data Masking can complement the design if specific columns such as phone numbers need to be obscured, but it is not a substitute for row-level authorization.
- Layer in least privilege, auditing, and encryption alongside RLS.

Memory rule: Row-Level Security controls which rows a user may see. Dynamic Data Masking controls how a sensitive column value appears. They solve different problems.

```
-- Row-Level Security example: restrict rows by region
CREATE FUNCTION dbo.fn_RegionSecurityPredicate(@RegionId AS NVARCHAR(10))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS fn_result
WHERE @RegionId = CAST(SESSION_CONTEXT(N'RegionId') AS NVARCHAR(10))
  OR IS_MEMBER('db_owner') = 1;
GO

CREATE SECURITY POLICY dbo.RegionAccessPolicy
ADD FILTER PREDICATE dbo.fn_RegionSecurityPredicate(RegionId)
ON dbo.Customers
WITH (STATE = ON);
```

```
-- Dynamic Data Masking example: mask a phone number column
ALTER TABLE dbo.Customers
ALTER COLUMN PhoneNumber ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-XXX-",4)');
```

Reference: Microsoft Learn, Dynamic Data Masking; Row-Level Security, learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking

12. Application DBA Duties and Change Management

Q23. How do you deploy database changes into production?

- Peer review and test every change in development and non-production first.
- Validate dependencies, locking and performance impact, permissions, deployment duration, and the rollback plan before submitting the change.
- The change request should include business reason, implementation steps, affected databases and objects, risk and impact, validation plan, rollback plan, expected duration, maintenance window, and approvals.
- After CAB approval, confirm backups and recovery readiness before the window opens.

- Execute in the approved window, capture deployment evidence, run smoke tests and technical validation, and monitor SQL Server and application health.
- Close the change only after validation succeeds. If validation fails, execute the predefined rollback plan rather than improvising in production.

Q24. How do you handle database roles and permissions?

- Follow least privilege as the default posture.
- Grant permissions to database roles, not directly to individual users. Add users and service accounts as members of the appropriate role.
- Separate read, write, execute, and administrative permissions clearly.
- Applications should not run under db_owner or sysadmin.
- Grant service accounts only what the application actually needs.
- Route all privileged changes through the organization's security and change management process, and keep them auditable.

Q25. What does an Application DBA do?

- Act as the bridge between development and application teams and the enterprise database and platform team.
- Own database deployments, schema changes, permissions, and SQL Agent jobs.
- Own application performance troubleshooting, indexes, and statistics.
- Own production change management, release support, and database migrations.
- Troubleshoot connectivity and query issues, and escalate infrastructure or platform problems to the enterprise DBA team when needed.
- The core differentiator versus an infrastructure DBA is understanding both the platform and how the application actually uses it.

Q26. How do you troubleshoot a slow SQL Agent job?

- Compare the current runtime against the job's historical baseline first.
- Identify the slow step and confirm whether it is T-SQL, SSIS, PowerShell, or a backup or restore process.
- For a T-SQL step, check the execution plan, Query Store history, blocking, waits, CPU and I/O, statistics, index usage, parameter behavior, transaction log growth, and tempdb pressure.
- Check for concurrent workload and data volume growth around the same time window.
- Identify what changed near the point the runtime increased: a deployment, an index change, a statistics update, a configuration change, server load, or data growth.
- Fix the root cause, test it outside production, deploy through change control, and monitor the next several executions.

13. Leadership and Team Experience

Q27. What size DBA team have you worked with?

- State the team size, for example roughly 35 to 40 people, across L1, L2, L3, and senior or onsite roles.
- Tie it to a skill: structured escalation, documentation, handoffs, and clear ownership during incidents and changes.

Q28. Have you managed a DBA team?

- State the team size you led, for example around ten people across different support levels.
- List responsibilities: workload allocation, technical guidance, escalations, production change review, mentoring junior DBAs, and incident coordination.
- For critical incidents, describe your priority order: assign clear owners, restore service first, communicate status, then run root-cause analysis and document preventive actions.

14. Quick Reference: Common Mistakes and Correct Answers

Topic	Incorrect answer	Correct answer
Standard Edition AG	Secondary can serve reads	Basic AG secondary has no read access or backup support
Bidirectional sync	Default to updating subscribers	Avoid updatable subscriptions for new development; design ownership and conflict handling explicitly
Regional row security	Use Dynamic Data Masking	Use Row-Level Security for row isolation; DDM only supplements column display
Performance troubleshooting	Start with Profiler	Start with Query Store, DMVs, and Extended Events
Transactional replication	Describe it as two way	It is Publisher to Subscriber unless a special topology is explicitly built
Transactional articles	Any table can be published	Published tables require a primary key
Snapshot replication PK	Primary key is mandatory	Snapshot replication does not require a primary key
Archiving 25 years of data	Start with partitioning	Start with retention rules, dependencies, and access requirements, then choose the mechanism
Azure migration	Start with backup and restore	Choose Azure SQL Database, Managed Instance, or Azure VM first, then the migration method
5 GB migration	Only bandwidth matters	Compatibility, dependencies, security, connectivity, and cutover all matter
Professional Edition	Guess what the interviewer means	Verify with SERVERPROPERTY('Edition')
Missing index suggestions	Create the index immediately	Validate against workload, actual plan, and Query Store before implementing

15. Priority Preparation Areas

- SQL Server replication concepts and article-level filtering.
- Always On limitations on Standard Edition versus Enterprise Edition.

- Azure migration architecture decisions: Azure SQL Database, Managed Instance, or Azure VM.
- SQL performance troubleshooting using Query Store, DMVs, and Extended Events.
- Data archiving and retention design before any technical mechanism.
- Row-Level Security versus Dynamic Data Masking for regional or tenant data isolation.
- Production change management and rollback discipline.

16. Reference List

- SQL Server to Azure SQL Managed Instance, Migration Guide, learn.microsoft.com/data-migration/sql-server/managed-instance/guide
- SQL Server to Azure SQL Managed Instance, Migration Overview, learn.microsoft.com/data-migration/sql-server/managed-instance/overview
- Snapshot Replication, learn.microsoft.com/sql/relational-databases/replication/snapshot-replication
- Publish Data and Database Objects, learn.microsoft.com/sql/relational-databases/replication/publish/publish-data-and-database-objects
- Enhance General Replication Performance, learn.microsoft.com/sql/relational-databases/replication/administration/enhance-general-replication-performance
- Frequently Asked Questions for Replication Administrators, learn.microsoft.com/sql/relational-databases/replication/administration/frequently-asked-questions-for-replication-administrators
- Best Practices for Monitoring Workloads with Query Store, learn.microsoft.com/sql/relational-databases/performance/best-practice-with-the-query-store
- Understand and Resolve SQL Server Blocking Problems, learn.microsoft.com/troubleshoot/sql/database-engine/performance/understand-resolve-blocking
- Basic Availability Groups for a Single Database, learn.microsoft.com/sql/database-engine/availability-groups/windows/basic-availability-groups-always-on-availability-groups
- Updatable Subscriptions for Transactional Replication, learn.microsoft.com/sql/relational-databases/replication/transactional/updatable-subscriptions-for-transactional-replication
- Specify How Changes Are Propagated, Transactional Replication, learn.microsoft.com/sql/relational-databases/replication/transactional/transactional-articles-specify-how-changes-are-propagated
- Dynamic Data Masking, learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking
- Editions and Supported Features of SQL Server, learn.microsoft.com/sql/sql-server/editions-and-components-of-sql-server